Evaluación Parcial N°2 — Infraestructura Serverless

Asignatura	INY1106 — Infraestructura de Aplicaciones II
Estudiante	Fernando Galdámez (fer.galdamez@duocuc.cl)
Proyecto	Image Metadata API (AWS S3, DynamoDB, Lambda Python 3.11, API Gateway, CloudWatch)
Región y Cuenta AWS	us-east-1 (N. Virginia) — Cuenta Lab: 792975503302 (Rol: LabRole)
Endpoint Productivo	https://9xl860sgs4.execute-api.us-east-1.amazonaws.com/prod

1. Fase 1: Almacenamiento (Amazon S3) y Base de Datos (Amazon DynamoDB)

Se aprovisionó mediante Terraform el bucket S3 único `my-image-storage-bucket-20f07620` con bloqueo de acceso público y la tabla NoSQL `ImageMetadata` en Amazon DynamoDB con clave de partición `image_id` (tipo String) bajo modalidad *On-Demand* (`PAY_PER_REQUEST`).

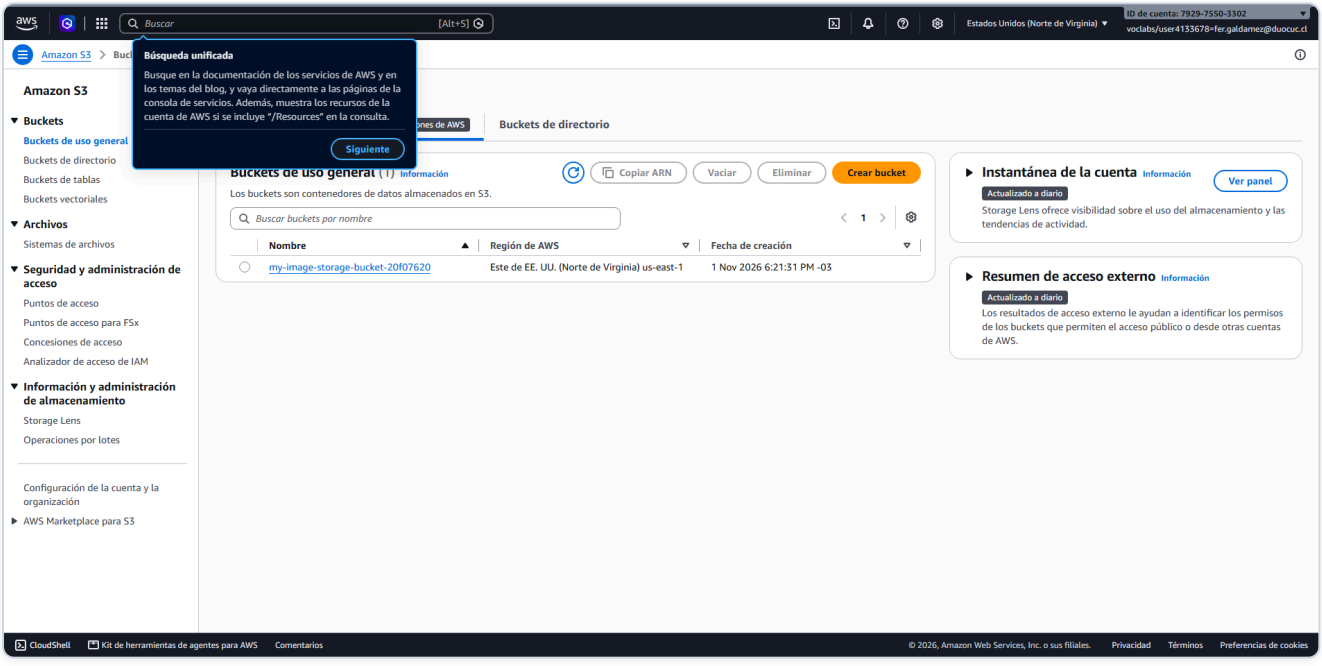


Figura 1.1: Bucket my-image-storage-bucket-20f07620 creado en Amazon S3 (us-east-1).

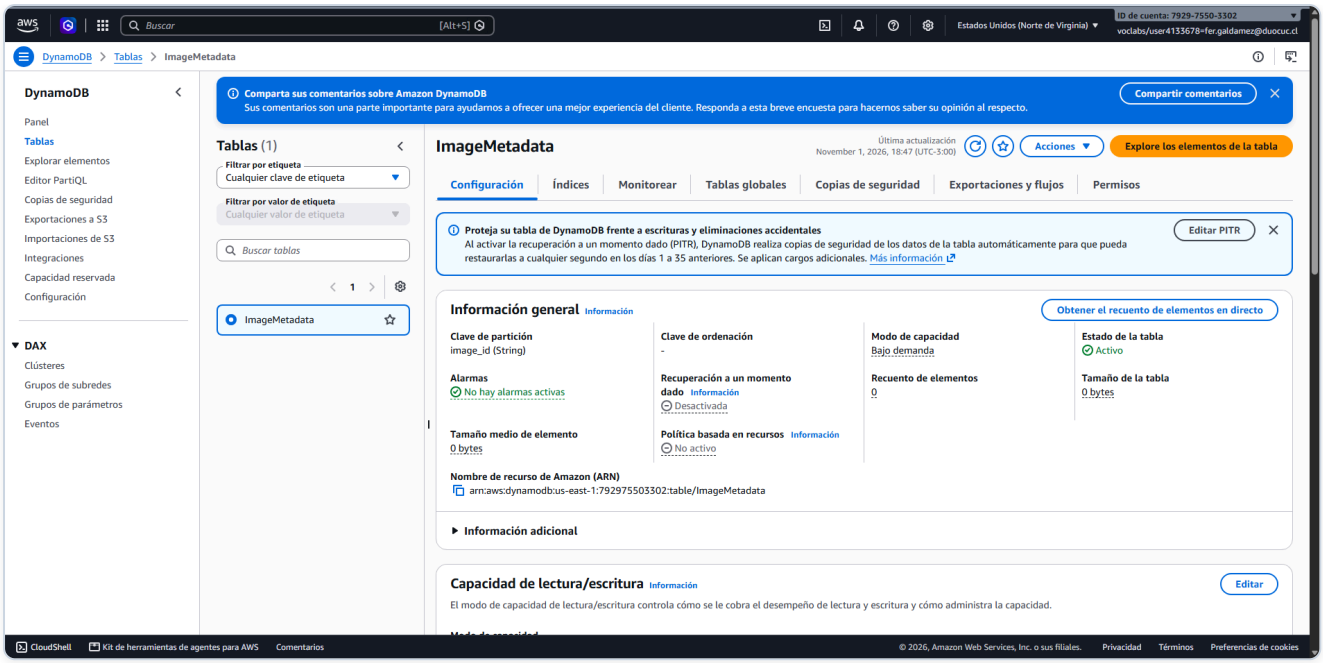


Figura 1.2: Tabla ImageMetadata activa con clave de partición image_id (String) y modo Bajo demanda.

2. Fase 2: Cómputo Serverless (Funciones AWS Lambda en Python 3.11)

Se desplegaron las funciones `upload_image` y `get_image_metadata` utilizando el runtime **Python 3.11**, un tiempo de espera (timeout) de **30 segundos**, rol de ejecución `LabRole` y variables de entorno parametrizadas.

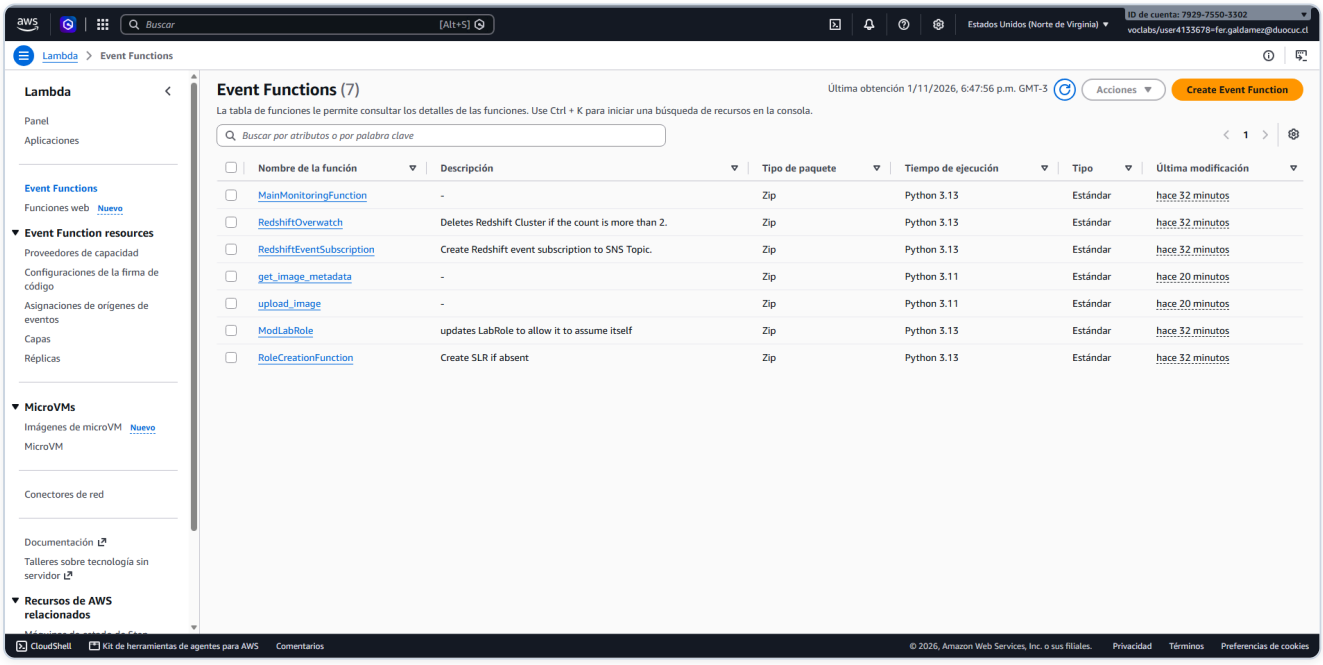


Figura 2.1: Funciones upload_image y get_image_metadata desplegadas en Python 3.11.

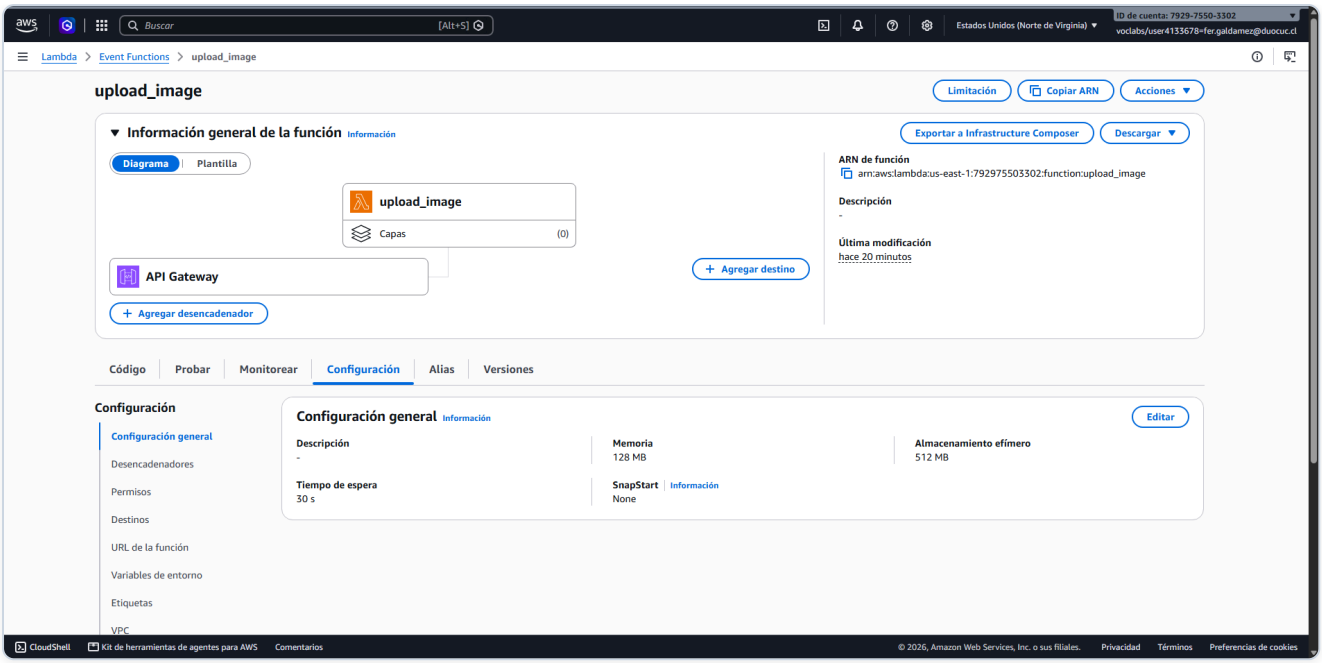


Figura 2.2: Configuración general de upload_image (Memoria 128 MB, Tiempo de espera 30 s).

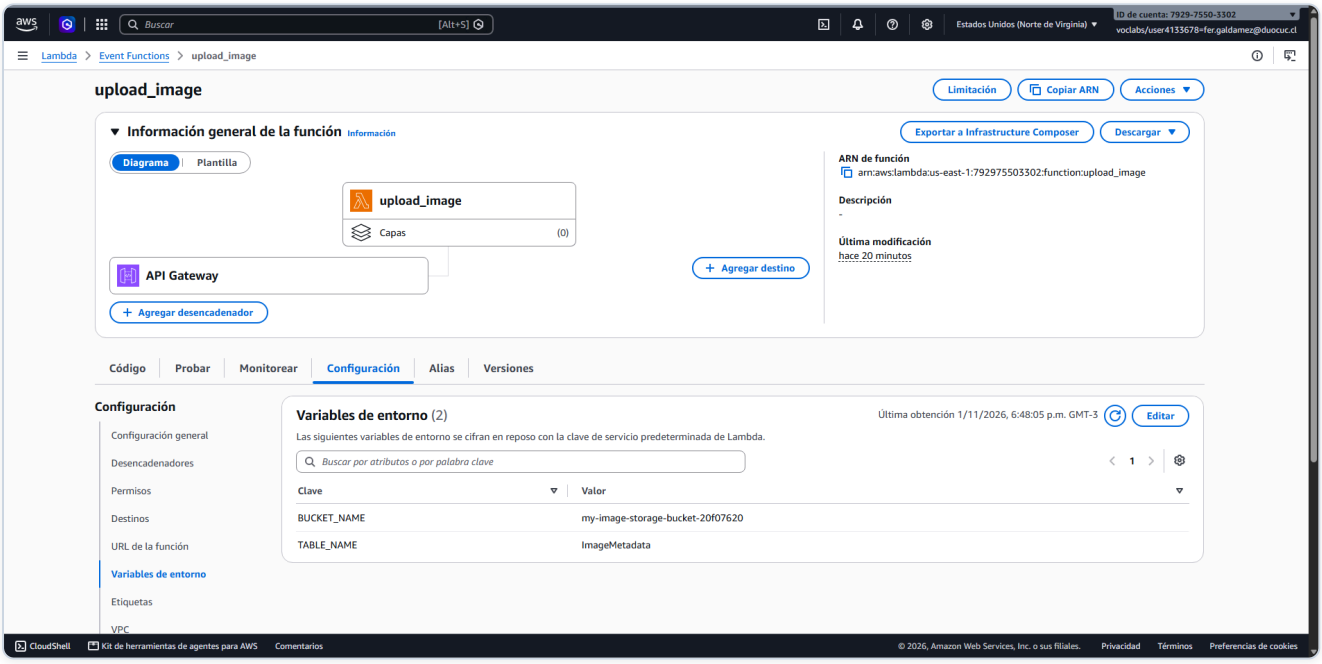


Figura 2.3: Variables de entorno BUCKET_NAME y TABLE_NAME en upload_image.

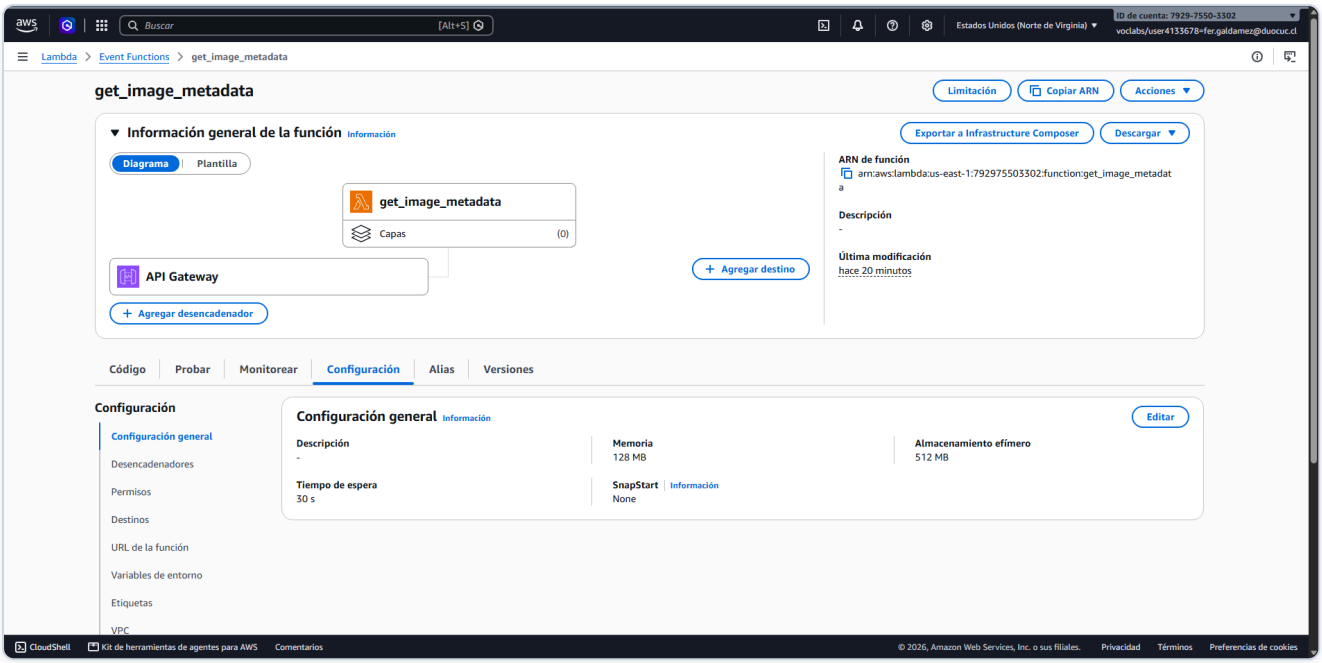


Figura 2.4: Configuración general de get_image_metadata (Memoria 128 MB, Tiempo de espera 30 s).

3. Fase 3: Configuración de AWS API Gateway

Se implementó la API REST regional `ImageMetadataAPI` (ID: `9xl860sgs4`) con soporte binario (`image/jpeg`, `*/*`), integrando los recursos `POST /upload` hacia `upload_image` y `GET /metadata/{image_id}` hacia `get_image_metadata` mediante integración `AWS_PROXY`, desplegada en la etapa `prod`.

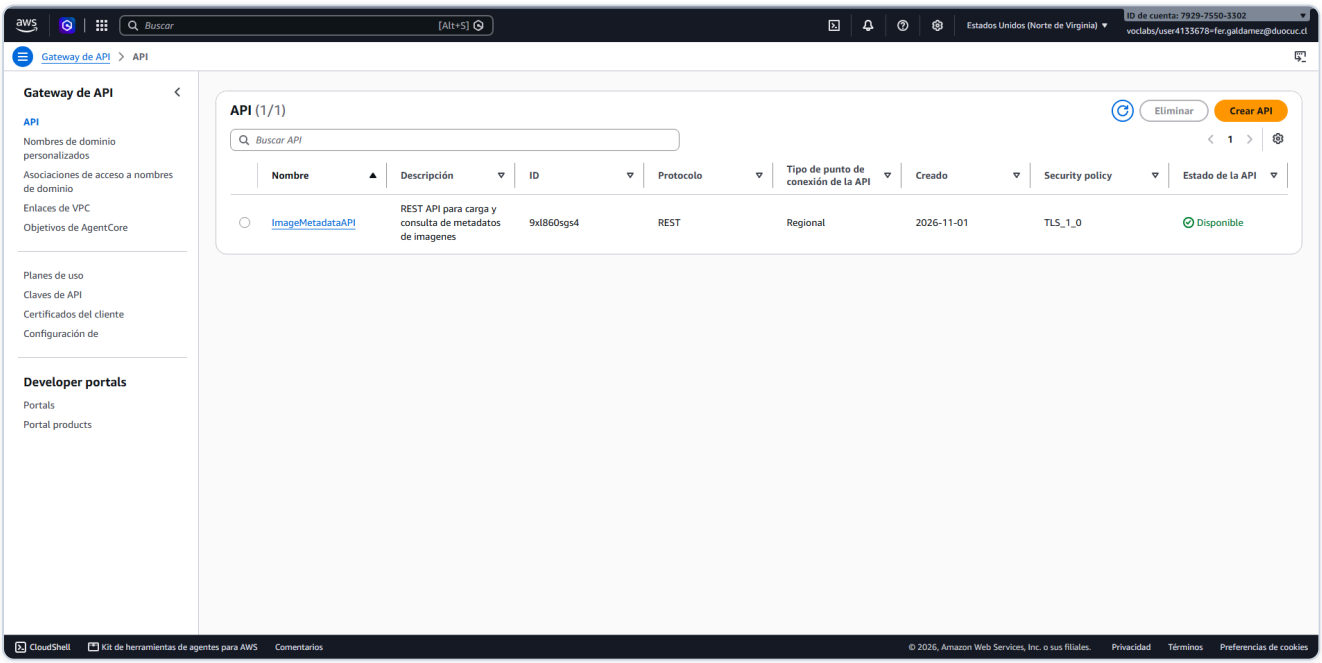


Figura 3.1: API REST ImageMetadataAPI registrada en Amazon API Gateway.

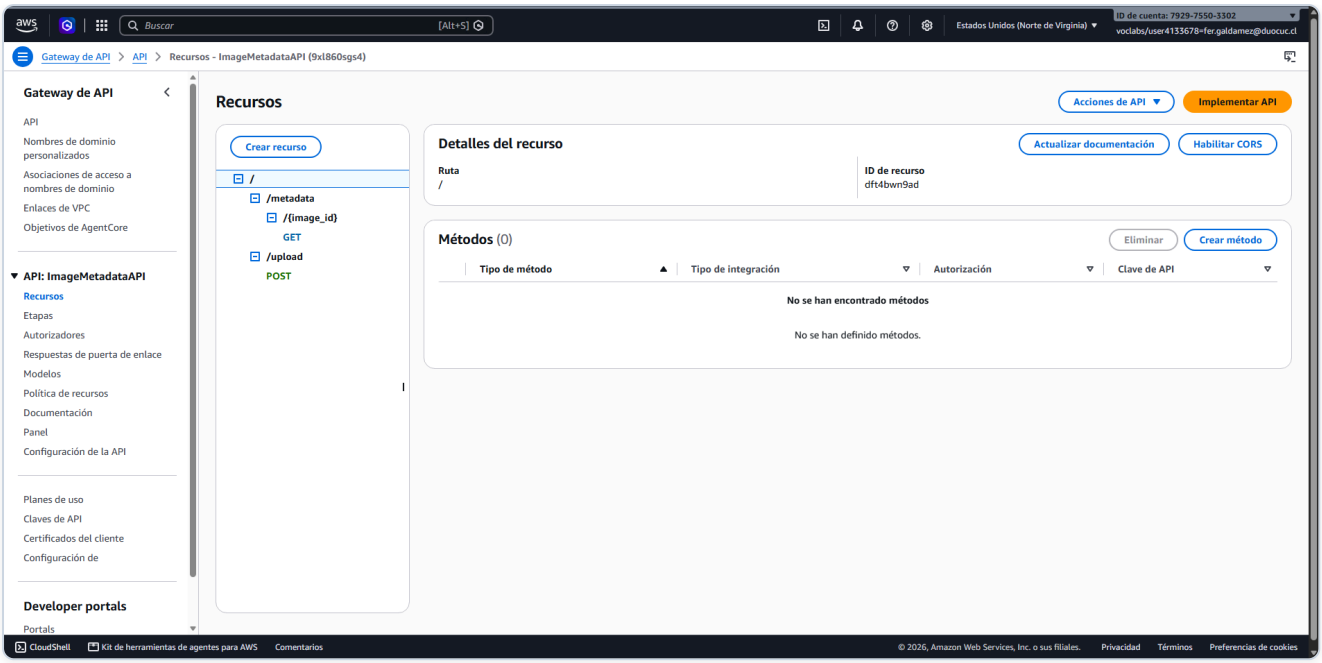


Figura 3.2: Árbol de recursos y métodos POST /upload y GET /metadata/{image_id}.

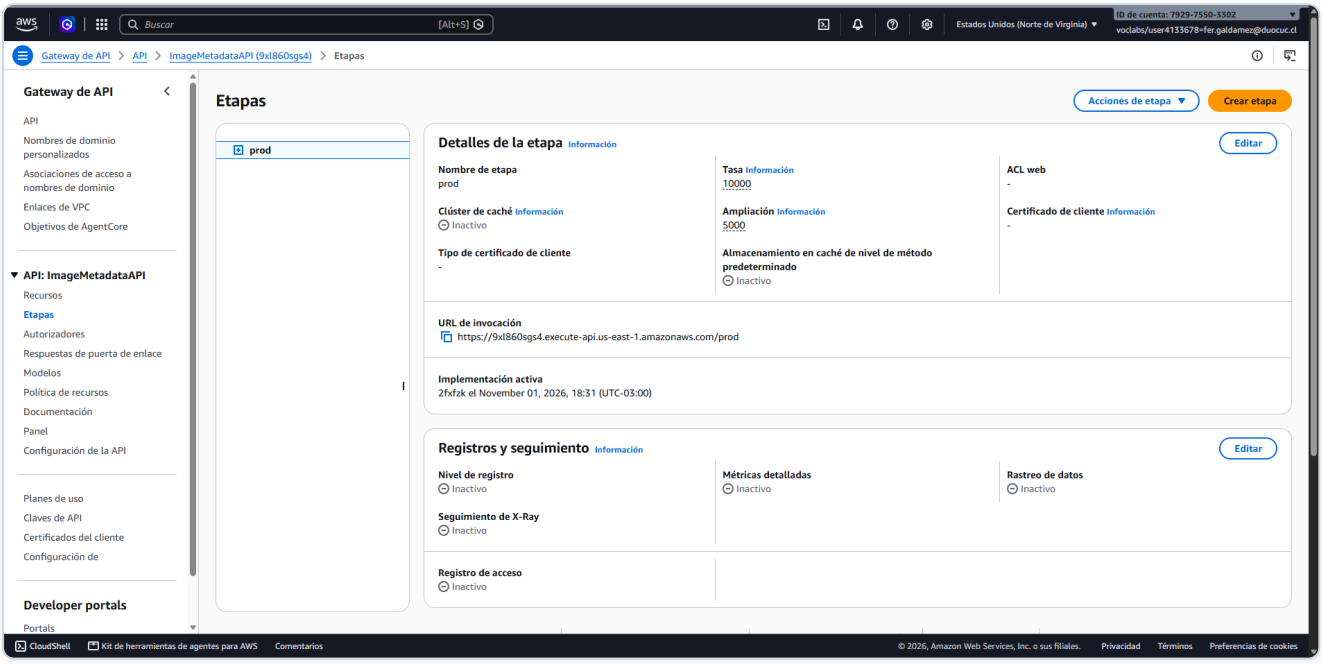


Figura 3.3: Etapa prod desplegada y URL de invocación activa.

4. Fase 4: Pruebas de Funcionamiento de la API (POST y GET)

4.1 Solicitud POST /upload (Subida de Imagen Binaria JPG)

```
curl -X POST https://9xl860sgs4.execute-api.us-east-1.amazonaws.com/prod/upload \
  -H "Content-Type: image/jpeg" \
  --data-binary "@test_image.jpg"

# Respuesta HTTP 200 OK:
{
  "image_id": "a26f5583-b891-4666-bc67-81cebac0319c",
  "bucket_name": "my-image-storage-bucket-20f07620",
```

```
  "s3_key": "images/a26f5583-b891-4666-bc67-81cebac0319c.jpg",
  "size_bytes": 137,
  "upload_date": "2026-11-01T21:34:53.570136+00:00"
}
```

4.2 Solicitud GET /metadata/{image_id} (Consulta de Metadatos)

```
curl -X GET https://9xl860sgs4.execute-api.us-east-1.amazonaws.com/prod/metadata/a26f5583-b891-4666-bc67-81cebac0319c

# Respuesta HTTP 200 OK:
{
  "image_id": "a26f5583-b891-4666-bc67-81cebac0319c",
  "bucket_name": "my-image-storage-bucket-20f07620",
  "s3_key": "images/a26f5583-b891-4666-bc67-81cebac0319c.jpg",
  "size_bytes": "137",
  "upload_date": "2026-11-01T21:34:53.570136+00:00"
}
```

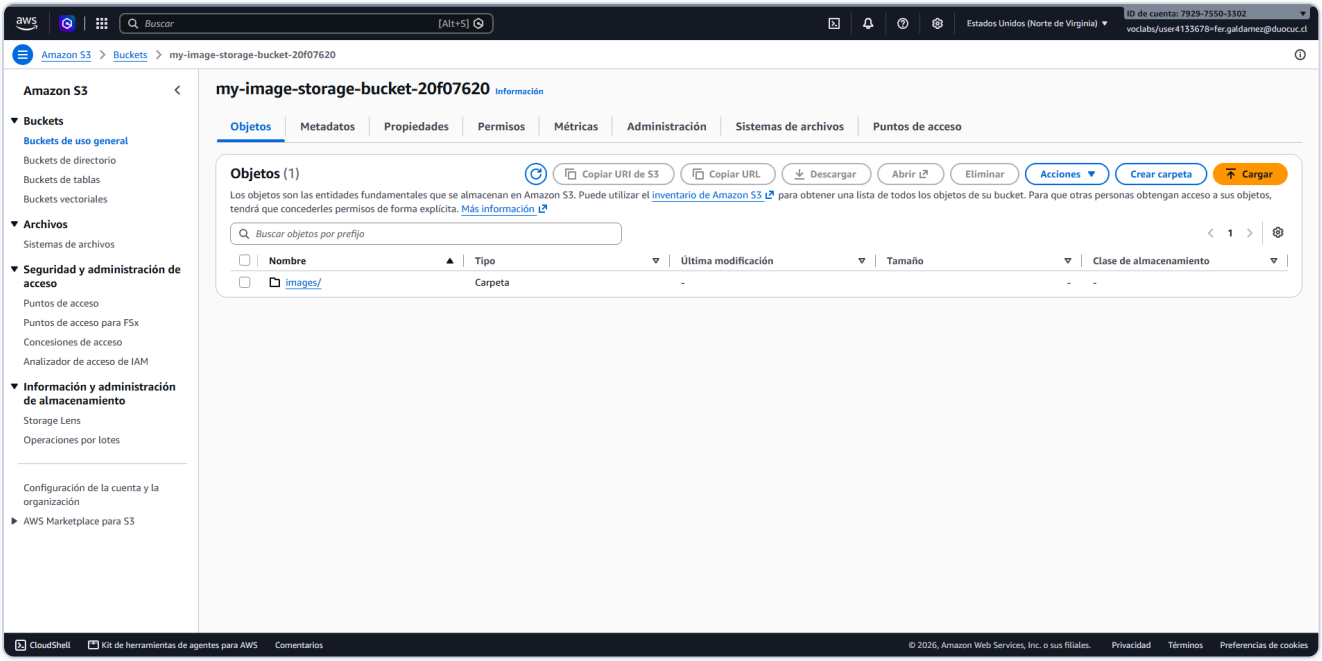


Figura 4.1: Prefijo images/ creado automáticamente en el bucket S3 tras la petición POST.

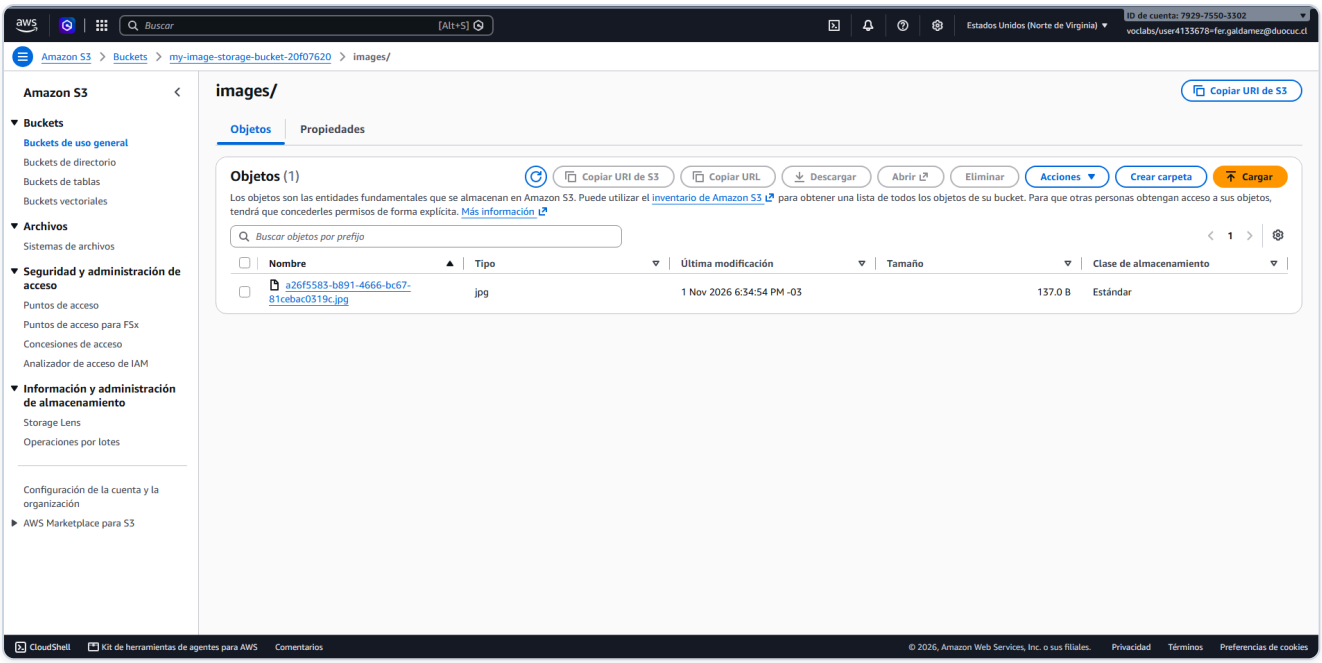


Figura 4.2: Imagen JPG almacenada en S3 con el UUID retornado por la API.

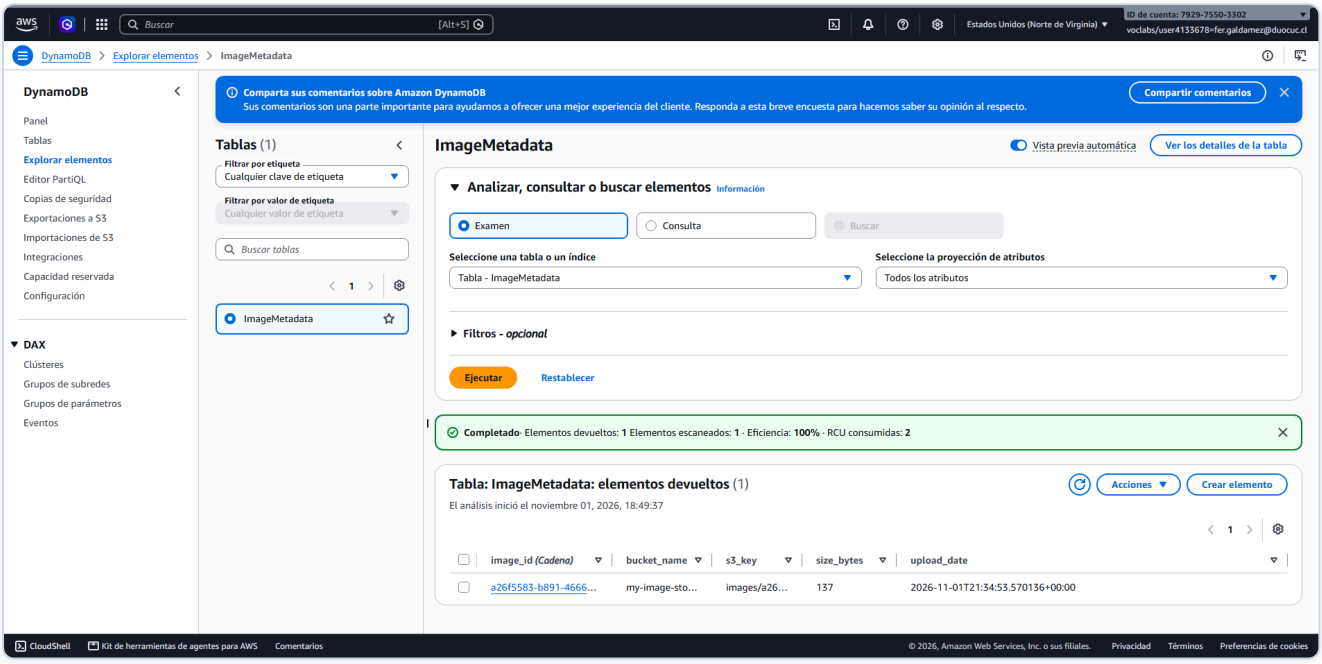


Figura 4.3: Metadatos persistidos y verificados en el explorador de elementos de DynamoDB.

5. Fase 5: Monitoreo, Troubleshooting y Optimización en CloudWatch

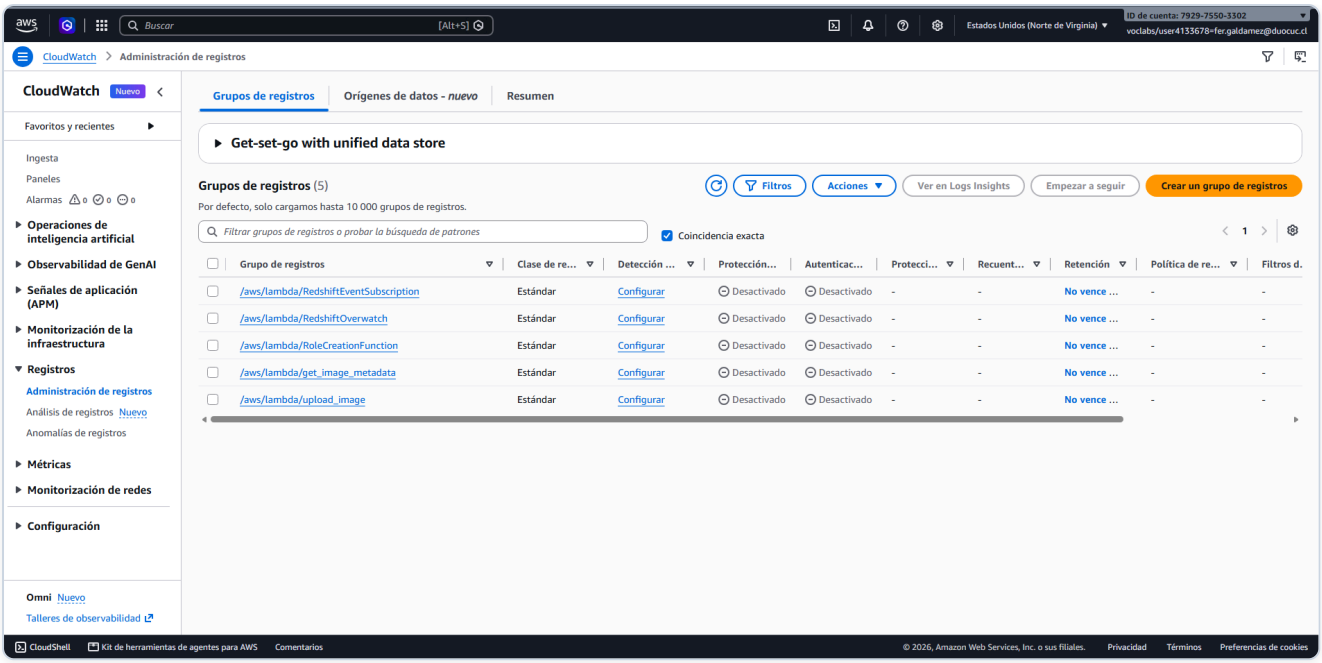


Figura 5.1: Grupos de logs /aws/lambda/upload_image y /aws/lambda/get_image_metadata en CloudWatch.

```
# Métricas REPORT extraídas de CloudWatch Logs:
/aws/lambda/upload_image:
REPORT RequestId: 32fcf9f8-4749-4e84-8e10-382c88b83780  Duration: 408.78 ms  Billed Duration: 925 ms
Memory Size: 128 MB  Max Memory Used: 94 MB  Init Duration: 515.88 ms

/aws/lambda/get_image_metadata:
REPORT RequestId: b34f7c4e-f7da-4d5a-a4d7-fa04418d1150  Duration: 260.39 ms  Billed Duration: 754 ms
Memory Size: 128 MB  Max Memory Used: 88 MB  Init Duration: 493.07 ms
```

Análisis de Optimización de Rendimiento y Escalabilidad

1. **Dimensionamiento de Memoria y CPU Proporcional:** AWS Lambda asigna capacidad de vCPU y red en proporción directa a la memoria configurada. Incrementar la memoria de 128 MB a 256 MB o 512 MB duplica/cuadruplica los ciclos de CPU disponibles para decodificar payloads Base64 y ejecutar escrituras TLS hacia S3 y DynamoDB, disminuyendo la latencia de respuesta sin incrementar significativamente el costo por invocación.
2. **Mitigación de Cold Starts y Concurrencia Provisionada:** Los registros muestran un *Init Duration* (~500 ms) en el arranque en frío inicial. Frente a incrementos de demanda, habilitar *Provisioned Concurrency* mantiene entornos pre-inicializados eliminando la latencia de arranque, mientras que *Reserved Concurrency* protege la cuota de la cuenta y evita estrangulamiento (throttling).
3. **Reutilización de Conexiones en Warm Context:** La instanciación global de `boto3.client('s3')` y `boto3.resource('dynamodb')` fuera del `lambda_handler` permite reutilizar el pool de conexiones HTTP/TLS entre ejecuciones consecutivas, reduciendo la latencia de red y optimizando el uso de infraestructura.

6. Anexo: Código Fuente Implementado

6.1 Infraestructura como Código (main.tf)

```
terraform {
  required_version = ">= 1.0.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

# Tabla DynamoDB
resource "aws_dynamodb_table" "image_metadata" {
  name         = "ImageMetadata"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "image_id"

  attribute {
    name = "image_id"
    type = "S"
  }

  tags = {
    Environment = "Lab"
    Project     = "ImageMetadataAPI"
  }
}

# Referencia al Bucket S3 existente
data "aws_s3_bucket" "image_bucket" {
  bucket = "my-image-storage-bucket-20f07620"
}

# LabRole existente del laboratorio
data "aws_iam_role" "lab_role" {
  name = "LabRole"
}

# Lambda 1: upload_image
resource "aws_lambda_function" "upload_image" {
  function_name    = "upload_image"
  role             = data.aws_iam_role.lab_role.arn
  handler          = "upload_image.lambda_handler"
  runtime          = "python3.11"
  timeout          = 30
  memory_size      = 128
  filename         = "${path.module}/lambdas/upload_image.zip"
  source_code_hash = filebase64sha256("${path.module}/lambdas/upload_image.zip")

  environment {
    variables = {
      BUCKET_NAME = data.aws_s3_bucket.image_bucket.bucket
```

```
      TABLE_NAME  = aws_dynamodb_table.image_metadata.name
    }
  }

  tags = {
    Environment = "Lab"
    Project     = "ImageMetadataAPI"
  }
}

# Lambda 2: get_image_metadata
resource "aws_lambda_function" "get_image_metadata" {
  function_name    = "get_image_metadata"
  role             = data.aws_iam_role.lab_role.arn
  handler          = "get_image_metadata.lambda_handler"
  runtime          = "python3.11"
  timeout          = 30
  memory_size      = 128
  filename         = "${path.module}/lambdas/get_image_metadata.zip"
  source_code_hash = filebase64sha256("${path.module}/lambdas/get_image_metadata.zip")

  environment {
    variables = {
      TABLE_NAME = aws_dynamodb_table.image_metadata.name
    }
  }

  tags = {
    Environment = "Lab"
    Project     = "ImageMetadataAPI"
  }
}

# ----------------- FASE 3: API GATEWAY REST API -----------------

resource "aws_api_gateway_rest_api" "image_api" {
  name        = "ImageMetadataAPI"
  description = "REST API para carga y consulta de metadatos de imagenes"

  binary_media_types = ["image/jpeg", "*/*"]

  endpoint_configuration {
    types = ["REGIONAL"]
  }

  tags = {
    Environment = "Lab"
    Project     = "ImageMetadataAPI"
  }
}

# 1. Endpoint /upload (POST)
resource "aws_api_gateway_resource" "upload_res" {
  rest_api_id = aws_api_gateway_rest_api.image_api.id
  parent_id   = aws_api_gateway_rest_api.image_api.root_resource_id
  path_part   = "upload"
}
```

```
resource "aws_api_gateway_method" "upload_method" {
  rest_api_id   = aws_api_gateway_rest_api.image_api.id
  resource_id   = aws_api_gateway_resource.upload_res.id
  http_method   = "POST"
  authorization = "NONE"
}

resource "aws_api_gateway_integration" "upload_integration" {
  rest_api_id             = aws_api_gateway_rest_api.image_api.id
  resource_id             = aws_api_gateway_resource.upload_res.id
  http_method             = aws_api_gateway_method.upload_method.http_method
  integration_http_method = "POST"
  type                    = "AWS_PROXY"
  uri                     = aws_lambda_function.upload_image.invoke_arn
}

# 2. Endpoint /metadata/{image_id} (GET)
resource "aws_api_gateway_resource" "metadata_res" {
  rest_api_id = aws_api_gateway_rest_api.image_api.id
  parent_id   = aws_api_gateway_rest_api.image_api.root_resource_id
  path_part   = "metadata"
}

resource "aws_api_gateway_resource" "image_id_res" {
  rest_api_id = aws_api_gateway_rest_api.image_api.id
  parent_id   = aws_api_gateway_resource.metadata_res.id
  path_part   = "{image_id}"
}

resource "aws_api_gateway_method" "get_metadata_method" {
  rest_api_id   = aws_api_gateway_rest_api.image_api.id
  resource_id   = aws_api_gateway_resource.image_id_res.id
  http_method   = "GET"
  authorization = "NONE"

  request_parameters = {
    "method.request.path.image_id" = true
  }
}

resource "aws_api_gateway_integration" "get_metadata_integration" {
  rest_api_id             = aws_api_gateway_rest_api.image_api.id
  resource_id             = aws_api_gateway_resource.image_id_res.id
  http_method             = aws_api_gateway_method.get_metadata_method.http_method
  integration_http_method = "POST"
  type                    = "AWS_PROXY"
  uri                     = aws_lambda_function.get_image_metadata.invoke_arn
}

# Permisos Lambda para API Gateway
resource "aws_lambda_permission" "api_upload_permission" {
  statement_id  = "AllowAPIGatewayInvokeUpload"
  action        = "lambda:InvokeFunction"
  function_name = aws_lambda_function.upload_image.function_name
  principal     = "apigateway.amazonaws.com"
  source_arn    = "${aws_api_gateway_rest_api.image_api.execution_arn}/*/*"
}
```

```
resource "aws_lambda_permission" "api_get_permission" {
  statement_id  = "AllowAPIGatewayInvokeGetMetadata"
  action        = "lambda:InvokeFunction"
  function_name = aws_lambda_function.get_image_metadata.function_name
  principal     = "apigateway.amazonaws.com"
  source_arn    = "${aws_api_gateway_rest_api.image_api.execution_arn}/*/*"
}

# Despliegue y Stage 'prod'
resource "aws_api_gateway_deployment" "api_deployment" {
  rest_api_id = aws_api_gateway_rest_api.image_api.id

  depends_on = [
    aws_api_gateway_integration.upload_integration,
    aws_api_gateway_integration.get_metadata_integration
  ]

  triggers = {
    redeployment = sha1(jsonencode([
      aws_api_gateway_resource.upload_res.id,
      aws_api_gateway_method.upload_method.id,
      aws_api_gateway_integration.upload_integration.id,
      aws_api_gateway_resource.metadata_res.id,
      aws_api_gateway_resource.image_id_res.id,
      aws_api_gateway_method.get_metadata_method.id,
      aws_api_gateway_integration.get_metadata_integration.id
    ]))
  }

  lifecycle {
    create_before_destroy = true
  }
}

resource "aws_api_gateway_stage" "prod_stage" {
  deployment_id = aws_api_gateway_deployment.api_deployment.id
  rest_api_id   = aws_api_gateway_rest_api.image_api.id
  stage_name    = "prod"
}

output "api_endpoint_url" {
  value = aws_api_gateway_stage.prod_stage.invoke_url
}
```

6.2 Función Lambda upload_image.py

```
import json
import os
import uuid
import base64
import datetime
import boto3

s3 = boto3.client('s3')
dynamodb = boto3.resource('dynamodb')

BUCKET_NAME = os.environ.get('BUCKET_NAME')
```

```
TABLE_NAME = os.environ.get('TABLE_NAME')
table = dynamodb.Table(TABLE_NAME)

def lambda_handler(event, context):
    try:
        is_b64 = event.get('isBase64Encoded', False)
        body = event.get('body', '')

        if is_b64:
            image_data = base64.b64decode(body)
        elif isinstance(body, str):
            image_data = body.encode('utf-8')
        else:
            image_data = body

        image_id = str(uuid.uuid4())
        s3_key = f"images/{image_id}.jpg"
        size_bytes = len(image_data)
        timestamp = datetime.datetime.now(datetime.timezone.utc).isoformat()

        # Subir a S3
        s3.put_object(
            Bucket=BUCKET_NAME,
            Key=s3_key,
            Body=image_data,
            ContentType='image/jpeg'
        )

        # Guardar metadatos en DynamoDB
        metadata_item = {
            'image_id': image_id,
            'bucket_name': BUCKET_NAME,
            's3_key': s3_key,
            'size_bytes': size_bytes,
            'upload_date': timestamp
        }
        table.put_item(Item=metadata_item)

        return {
            'statusCode': 200,
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            },
            'body': json.dumps(metadata_item)
        }
    except Exception as e:
        return {
            'statusCode': 500,
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            },
            'body': json.dumps({'error': str(e)})
        }
```

6.3 Función Lambda get_image_metadata.py

```
import json
import os
import boto3

dynamodb = boto3.resource('dynamodb')
TABLE_NAME = os.environ.get('TABLE_NAME')
table = dynamodb.Table(TABLE_NAME)

def lambda_handler(event, context):
    try:
        path_params = event.get('pathParameters') or {}
        image_id = path_params.get('image_id')

        if not image_id:
            return {
                'statusCode': 400,
                'headers': {
                    'Content-Type': 'application/json',
                    'Access-Control-Allow-Origin': '*'
                },
                'body': json.dumps({'error': 'Falta el parámetro image_id'})
            }

        response = table.get_item(Key={'image_id': image_id})
        item = response.get('Item')

        if not item:
            return {
                'statusCode': 404,
                'headers': {
                    'Content-Type': 'application/json',
                    'Access-Control-Allow-Origin': '*'
                },
                'body': json.dumps({'error': 'Metadatos de imagen no encontrados'})
            }

        # Convertir tipos no serializables si fuera necesario
        return {
            'statusCode': 200,
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            },
            'body': json.dumps(item, default=str)
        }
    except Exception as e:
        return {
            'statusCode': 500,
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*'
            },
            'body': json.dumps({'error': str(e)})
        }
```